

From Prompt to State: Verifiable Grounding and Operative Replay for LLM-Mediated Control

Tomas Laurenzo

Department of Critical Media Practices

University of Colorado Boulder

Boulder, Colorado, USA

tomas@laurenzo.net

Abstract—Applications that let a large language model control software contain a consequential step that prompt logs do not explain by themselves: model text must be interpreted as an action and executed by ordinary code. This paper isolates that language-to-state boundary and makes it auditable. We define a trace contract that links the invocation assembled by the application, the model response or a commitment to it, the command produced by grounding, the domain rules, and the resulting state and semantic events. A verifier reconstructs the invocation when text is retained, checks that the response grounds to the recorded command, and replays the transition under frozen rules. We implement the contract in BatLLM, a turn-based game in which local language models translate human instructions into a small command language. Across 60 generated sessions and 1080 transitions, every state and event sequence replayed equivalently. Production semantics agreed with a separate executable reference in 5000 cases; the verifier rejected 1560 re-anchored semantic perturbations and accepted 180 semantically equivalent serialisations. The contribution is a bounded method for explaining how model text became application state. It does not require reproducing the model response itself.

Index Terms—large language models, provenance, replay, grounding, verification, human-AI interaction

I. INTRODUCTION

Consider an LLM-controlled game that records a human instruction, the response `C90`, and a bot whose heading has changed by ninety degrees. The record still leaves several questions unanswered. Which messages and state were submitted to the model? Did the returned text actually ground to `C90`? Which rules interpreted that command? Do those rules entail the recorded state and events? Repeating the model call cannot answer these questions: inference is probabilistic and can also vary numerically across execution environments [1].

The difficulty comes from treating an LLM-mediated action as a single operation. It is instead a composition of two computational regimes. A model produces text under uncertain inference; an application then parses that text and applies conventional program semantics. Prompt-response logs describe the first regime. Action or state logs describe the second. Auditing the action requires evidence that binds them.

We study this boundary through BatLLM [4], an open-source turn-based game in which two human players influence software agents through natural-language instructions. A local LLM returns a command; deterministic game code applies movement, rotation, shielding, shooting, collision, and damage rules. The

command language is deliberately small, but the resulting transitions are non-trivial enough to expose the difference between linguistic evidence and executable consequence.

This paper makes three contributions. First, it defines a trace contract and four verification predicates that distinguish trace integrity, invocation reconstruction, response-to-command grounding, and operative replay. Second, it implements the contract as a versioned BatLLM research runtime with explicit text-retention profiles and a separately written executable semantics. Third, it evaluates the implementation through replay, differential testing, semantic perturbation, benign-serialisation controls, and storage measurement. The object of verification is intentionally bounded: the paper accounts for how a retained response became a domain state, not whether a model can regenerate that response.

II. RELATED WORK

Games and interactive environments provide structured settings for observing LLM agents. AgentBoard measures progress over multi-turn tasks [2], while Cradle studies general computer control through screen-based interaction [3]. BatLLM differs in purpose: the game is used here as an inspectable domain for verifying the transition from language to executable state.

Provenance standards offer a broader vocabulary for this problem. W3C PROV models entities, activities, agents, and derivations [5]. OpenTelemetry’s generative-AI conventions specify model input and output message attributes and warn that captured content may be sensitive [6]. PROV-AGENT extends provenance to prompts, responses, decisions, and workflow lineage [7]; a recent survey organises evidence tracing and execution provenance across observability, tools, memory, audit, and recovery [8].

Record-and-replay work addresses adjacent objectives. AgentRR reuses successful trajectories as later experience [9]. Prompt Scene Investigation records evidence for behavioural reconstruction [10]; *agrepl* replays captured external interactions without outbound access [11]. Paduraru *et al.* combine message-action traces, executable contracts, replay, and fault injection for agent orchestration [12]. These systems establish substantial prior art in provenance and replay. Our narrower verification object is the conjunction of an application-visible invocation, its linguistic response, the command grounded

TABLE I
EVIDENCE AVAILABLE BY TEXT-RETENTION PROFILE. “COMMIT.” DENOTES
A SHA-256 COMMITMENT WITHOUT RETAINED TEXT.

Predicate	Full	Redacted	Hash-only
V1 trace	yes	yes	yes
V2 invocation	reconstruct	struct.+hash	hash
V3 grounding	verify	no	no
V4 replay	verify	verify	verify

from that response, and the deterministic domain transition that follows.

III. A CONTRACT FOR THE LANGUAGE-TO-STATE BOUNDARY

A. Execution model

For step i , let u_i be the human instruction, h_i the ordered conversation history, s_i the pre-action state, θ_i the declared model configuration, and ρ_i the frozen domain rules. The application constructs the invocation record visible at its provider-adapter boundary,

$$a_i = A(u_i, h_i, s_i, \theta_i), \quad (1)$$

and receives response text y_i from provider M . A parser grounds the text into command c_i ; a transition function produces the next state and ordered semantic events:

$$c_i = P(y_i), \quad (s_{i+1}, e_i) = \delta(s_i, c_i; \rho_i). \quad (2)$$

The trace unit is

$$T_i = \langle u_i, h_i, s_i, a_i, \theta_i, y_i, c_i, \rho_i, e_i, s_{i+1} \rangle. \quad (3)$$

The adapter record contains the model, ordered messages, options, and stream setting passed by BatLLM, together with declared provider and endpoint. It is not a capture of provider-library internals, HTTP bytes, hidden defaults, model weights, or serving binaries.

B. Verification predicates

The contract separates four questions.

V1—Trace validity. Are the schema, identifiers, sequence, state continuity, final states, commitments, and ordered play chain internally consistent?

V2—Invocation evidence. Does the retained evidence support the invocation attributed to the application? With full text, the verifier reconstructs the user message and history from separately recorded prompt, state, system instructions, and context policy, then compares them with a_i .

V3—Grounding consistency. When response text is retained, does re-applying P produce the recorded command and validity status? Provider failures are represented separately and resolve operatively to ERR.

V4—Operative replay. Does re-evaluating $\delta(s_i, c_i; \rho_i)$ reproduce both s_{i+1} and e_i ? BatLLM compares discrete state exactly and position and rotation with absolute tolerance 10^{-6} .

Table I makes the disclosure trade-off explicit. Full retention supports all four checks. Redacted and hash-only traces

preserve structural evidence and operative replay, but cannot independently establish which command hidden response text would produce. Hashes are consistency commitments, not confidentiality or authentication: low-entropy text can be guessed, and a party able to replace a complete trace and every external copy can recompute all internal values.

A successful verification therefore has a precise meaning. Assuming deterministic P and δ and the semantics identified by the trace, V3 and V4 establish that the recorded command equals $P(y_i)$ and that the recorded state and events equal $\delta(s_i, P(y_i); \rho_i)$. Historical authenticity additionally requires a trusted recorder or an independently preserved signature, timestamp, or commitment. Verification does not imply that another invocation of M would reproduce y_i .

IV. IMPLEMENTATION

BatLLM normalises model text into movement (M , Md), clockwise and counter-clockwise rotation ($C\alpha$, $A\alpha$), shooting (B), and shield operations (S , $S1$, $S0$); unsupported text resolves to ERR. Gameplay is implemented as a pure transition module independent of the Kivy interface. The replay projection includes each bot’s health, position, rotation, and shield state. Prompt and diagnostic fields remain invocation evidence rather than gameplay state.

The schema-v3 research runtime is a headless path that uses the production parser and transition engine. Immediately before an adapter call, it materialises the ordered messages, declared provider and endpoint, requested model, generation options, and stream setting. It then records the response or error, normalised command, frozen rules, pre- and post-state, ordered semantic events, timestamps, latency, and retry count. Failed attempts are removed from later conversation history, matching BatLLM’s rollback policy. The reference evaluation uses a scripted provider to isolate trace semantics from model and network variability; a Modelito/Ollama adapter supplies a live local-model integration path.

Canonical JSON uses UTF-8, ordered keys, compact separators, and finite numbers. Separate commitments cover protected text, retained requests, transitions, play records, the ordered play chain, and the session envelope. The verifier reports predicate-specific failures and exits non-zero. To avoid evaluating replay only by invoking production code twice, the artefact also contains an executable reference semantics that imports no production gameplay module. It separately implements command parsing, movement bounds, rotation, shields, projectile traversal, collision, damage, state normalisation, and semantic events. This is differential evidence rather than a formal proof: both implementations can still share a specification error. Source, schema, experiments, and build instructions are available in the repository under `research/urucon2026` on main.

V. EVALUATION

A. Protocol

The evaluation asks whether valid traces satisfy the evidence available under each retention profile; whether production and reference semantics agree; whether the verifier distinguishes

TABLE II
REFERENCE ARTEFACT RESULTS.

Measure	Result
Schema-valid / semantically valid	60 / 60
State / event replays	1080 / 1080
Full invocation reconstructions	360
Retained-text groundings verified	360
Differential state/event/path cases	5000/5000/5000
Re-anchored perturbations rejected	1560/1560
Benign serialisations accepted	180/180
Raw bytes/play: full/redacted/hash	4374/4741/2203
Mean gzip bytes/play	439

semantic inconsistency from harmless representation changes; and what storage cost the trace representation imposes.

The corpus contains 60 sessions and 1080 transitions: twenty sessions per retention profile and eighteen transitions per session. It varies shared and independent histories, augmented and unaugmented prompts, game parameters, every documented command form, and invalid text. Responses are scripted. This design tests the trace and transition contract, not LLM competence, provider fidelity, or pedagogical effectiveness. All 60 traces validate against the published JSON Schema Draft 2020-12 with date-time format checking.

Differential testing uses a fixed seed and 5000 targeted and random cases spanning positions, rotation, health, shields, rules, commands, invalid text, and projectile outcomes. It compares command normalisation, resulting state, ordered events, and projectile path between production and reference implementations.

The perturbation experiment changes eleven classes of evidence: command, retained response, retained invocation, human prompt, state supplied to the model, rules, pre-state, post-state, events, deletion, and reordering. Applicable play-local mutations are placed early, midway, and late in sessions; rule changes are session-level. Before verification, every commitment derivable from mutated retained content is recomputed. A control reserialises each valid trace as compact ASCII, pretty Unicode, and recursively reversed key order. Storage is measured on canonical UTF-8 JSON and gzip level 9.

B. Results

Every session satisfied the schema and the semantic checks available under its retention profile (Table II). All 1080 recorded states and event lists replayed equivalently. Full retention reconstructed 360 adapter-boundary invocation records and verified 360 response-to-command mappings; the 720 reduced-retention transitions remained replayable without being reported as independently grounded.

Production and reference semantics agreed in all 5000 cases for normalisation, state, events, and projectile paths. The verifier rejected all 1560 applicable re-anchored perturbations and accepted all 180 benign variants. These are coverage results for the declared fixtures, not estimates of detection probability in deployment. Raw storage was 4374, 4741, and 2203 bytes

per play for full, redacted, and hash-only retention respectively; compression reduced the overall mean to 439 bytes per play.

VI. DISCUSSION

The contract turns one vague reproducibility question into three answerable audit questions: what did the application submit, how was the returned language grounded, and did the grounded action entail the recorded consequence? This separation matters because a replayable command log is not evidence of what a model said, while a prompt-response log is not evidence of what software subsequently did. Full traces bind both; reduced-retention traces state explicitly which part of that relation has been surrendered.

The design transfers to systems in which model text crosses a visible grounding boundary into a command language and the subsequent semantics can be frozen or otherwise reconstructed. Tool-calling interfaces, workflow languages, simulations, and constrained robotic controllers may meet this condition, but each domain needs its own state projection, event vocabulary, and independent semantic tests. BatLLM demonstrates the pattern in one bounded domain; it does not establish cross-domain generality.

The distinction between semantic consistency and provenance remains essential. Re-anchored perturbations show that the verifier checks relationships among fields rather than stale hashes alone, and benign serialisations show that acceptance is semantic rather than byte-for-byte. A comprehensive rewrite can nonetheless remain internally valid if every external anchor is also replaced. External chain of custody is therefore complementary to, not supplied by, operative replay.

VII. LIMITATIONS AND CONCLUSION

The evaluation uses scripted responses and a deterministic game domain. It does not measure live-provider completeness, model quality, human usefulness, or learning. Systems with stochastic post-grounding behaviour must additionally record random state or external effects. The separate reference semantics reduces circular testing but cannot exclude shared specification errors. Adapter records do not capture transport bytes or authenticate model weights and serving code. Reduced retention prevents independent grounding verification, and the graphical application still exports schema v2; the claims concern the explicit v3 research runtime.

Within these boundaries, the work establishes a useful result: model generation need not be repeatable for an action derived from a recorded response to be verifiable. By preserving evidence at the application boundary, exposing grounding as a checkable operation, and replaying explicit domain semantics, BatLLM gives a concrete account of how human instruction became computational state.

ACKNOWLEDGMENT

BatLLM was supported by the University of Colorado Boulder’s Center for Humanities & the Arts and Research & Innovation Office.

REFERENCES

- [1] J. Yuan *et al.*, “Understanding and mitigating numerical sources of nondeterminism in LLM inference,” in *Advances in Neural Information Processing Systems*, vol. 38, 2025.
- [2] C. Ma *et al.*, “AgentBoard: An analytical evaluation board of multi-turn LLM agents,” in *Advances in Neural Information Processing Systems*, vol. 37, 2024.
- [3] W. Tan *et al.*, “Cradle: Empowering foundation agents towards general computer control,” in *Proc. 42nd Int. Conf. Machine Learning*, 2025, pp. 58658–58725.
- [4] T. Laurenzo, “BatLLM: Local-first human–LLM-mediated play and analysis,” open-source software, version 0.3.6, 2026.
- [5] W3C Provenance Working Group, “PROV-O: The PROV ontology,” W3C Recommendation, 2013.
- [6] OpenTelemetry Authors, “Generative AI semantic conventions,” OpenTelemetry Specification, version 1.43.0, 2026.
- [7] R. Souza *et al.*, “PROV-AGENT: Unified provenance for tracking AI agent interactions in agentic workflows,” in *Proc. IEEE Int. Conf. eScience*, 2025, pp. 467–473.
- [8] Y. Wang *et al.*, “From agent traces to trust: A survey of evidence tracing and execution provenance in LLM agents,” arXiv:2606.04990, 2026.
- [9] E. Feng *et al.*, “Get experience from practice: LLM agents with record & replay,” arXiv:2505.17716, 2025.
- [10] L. F. Ledjaki, Y.-C. Chang, M. Loutis, and E. Aïmeur, “Prompt scene investigation: Uncovering the evidence in LLMs,” in *Companion Proc. ACM Web Conf.*, 2026, pp. 284–290.
- [11] R. Mudasiru, “Deterministic replay for AI agent systems,” arXiv:2607.16200, 2026.
- [12] C. Paduraru, P.-L. Bouruc, and A. Stefanescu, “A trace-based assurance framework for agentic AI orchestration: Contracts, testing, and governance,” in *Proc. 21st Int. Conf. Evaluation of Novel Approaches to Software Engineering*, 2026, pp. 420–427.